Definitions

A developer is an AI agent or human that is writing application code or configuations, debugging issues in the code or configuations, testing and validating results, and checking the code or configurations into a source code repository, like GitHub.

A README.md file describes the organization of the module or component in which the READM.md file resides.

A transient documentation file is an informatioal file created by an AI for focused diagnostic, planning, or temporary communication purposes to communicate with a human. These documenttion file summarize the state of a set of uncommited changes, document the rationale of a set of changes, provide go-foward options to a human, debugging notes, or document the set of changes in a commit. Once the functional chagnes are commited to a repository, the value of any transient documentation decreases substantially, and ultimately becomes noise. or in many cases are scripts.

A risky command is one that makes large, irreparable changes, like deletion of a large directory or updating a large amount of data in a database. Commands that are not risky include any read-only or diagnostic type commands.

Timless (reference "The Timeless Way of Building" by Christopher Alexander) is not an architectural style but a *state of being* achieved when an element is truly alive, whole, and in profound harmony with the actors, actions, and other elements that occur there. "Timeless" usually refers to:

- Enduring Appeal: Designs rooted in fundamental principles (like proportion, balance, and quality) that transcend fleeting trends and remain aesthetically pleasing for generations.
- Quality and Durability: Use of natural, honest materials (stone, wood, brick) and craftsmanship that age gracefully, gaining character rather than deteriorating.
- Functionality and Adaptability: Spaces that are flexible, functional, and can evolve to meet the changing needs of the people who use them, ensuring long-term relevance.

Statements

Documentation

1. Developers should document any top-level modules (i.e. directories)
2. Developers may document any level two directories should the complexity merit the documenation

3. An AI must read all the README files.
4. There must be a README in each top-level directory and in some of the subdirectories, unless that directory is ignored by Git.
5. There may be a README.md in a subdirectory of a top-level directory, should the complexity of the subdirectory merit additional documentation.
6. An AI must read any documentation in the docs directory.

README.md Files

1. A README.md document should cover:
2. README.md documenation should be timeless focusing on the current state of the implemenation
3. The purpose of the directory
4. The organization of the directory exlusive of README.md files in subdirectories
5. The high-level architecture of any components that the source code in the directory creates
6. Any inputs or preconditions required by the directory
7. The main entrypoints into or functions of the directory
8. References to other relevant documentation files
9. README.md files must not cover the directory organization of README.MD files for other directories, especialy nested directories of the README.md file's directory

Long Term Documentation Files

1. Developers must place documentation that is long term (i.e. timeless) that speaks to the architecture or operations of the capabilities in the project should be placed into the docs directory or one of the README.md files in the directories
2. Before an AI creates a new documentation file, the AI must look to see if there is a elevant, existing documentation file in which to place the new information.
3. A developer may create a top-level a top-level README to cover the orgainization of the project

Transient documentation files

1. A developer should create a top-level tmp directory for each project
2. A developer must add the tmp directory to the .gitignore file
3. Developers must create transient documentation files in the tmp directory
4. Developers must not commit transient files to a Git repository

5. Transient documentation fles must be named as:

   YYYYMMDD_HHMM_<name>

Where:
1. YYYYYMMDD_HHMM is the four-digit year, two-digit month and day, and the time in minutes in a 24-hour format 1. <name> is an AI or human assigned name for the file

Debugging and Testing

1. For a paritcular issue or enhancement, if there is a standard or agreed upon set of tests, the AI should run the test to reproduce the issue
2. If the issue is reproduced the AI should diagnose and then fix the root cause of the issue
3. Once the issue is fixed, the AI must run the tests and check for the issue or new issues 2.When restarting any services (e.g. Docker or Kubernetes), the AI must check for the availability of the relevant services.
4. The AI should use health checks to test for the availability of a service
5. The AI may use container commands (e.g. docker compose ps) to check for the avilabilty of a container in the absence of a defined health check
6. Once the developer remediates an issue, the developer must validate that the remediation rectifies the issue.
7. If there is a test suite or test that exhibits the issue, the developer must re-run the test.

Git

1. Developers must not commit changes until they are tested
2. An AI must not commit changes until the human developer aligns with the AI on completion of the test results.
3. When restructuring directories in a Git project, developers must use "git mv" over "mv" in order to preserve the Git history of the original file.

AI ChatBot

1. An AI should execute potentially long running commands or commands which may hang within a timeout command
2. An AI should prompt the human before executing a risky command
3. An AI should prompt the human before implementing significant architecture or configuration changes

Operational Commands

1. Operations staff must not pipe files into ansible commands

2. Configuration files must be deployed to target (or managed) hosts and used on the managed host
3. No staff should explicitly put passwords in commands lines, where they may be logged
4. Staff may use environment variables that contain passwords in a development environment
5. Developers should Playbooks (or updated playbooks) over direct commands, in order to establish a more mature repeatable process.